Exercise Sheet 11

The Vault Beneath

House rules.

- Nothing to install: `sqlite3` comes with Python. The Vault is one file, `data/guild.db`, with four tables: heroes, quests, party, regions.
- One file per exercise in `my_work/`, run from the lesson folder: `python my_work/ex1_knock_knock.py`.
- Every exercise starts with the **opener** (type it, don't paste it). Write each query in a triple-quoted string so it can span lines:

```
import sqlite3

con = sqlite3.connect("data/guild.db")
rows = con.execute("""
    SELECT ...
""").fetchall()
```

- `.fetchall()` gives a list of tuples (one per row), `.fetchone()` just the first tuple. Print with f-strings exactly as shown; the spacing is part of the answer.
- Want to look inside the Vault with your eyes? **DB Browser for SQLite** (free, sqlitebrowser.org) opens `guild.db` like a workbook. Optional; don't edit the data with it.
- Hand in: commit and push to `lesson-11`, then open a pull request. Tell Lars which SQL word you like best.

Exercise 1 Knock on the Vault

5 min · `ex1_knock_knock.py`

> **Raven from Orrin of the Clockworks:** *"The key is on the hook by the stairs. Mind the third step; it's a mimic. Once you're in, tell me what you see, so I know the Vault still knows you."*

(a) The Vault keeps a list of its own tables: `SELECT name FROM sqlite_master WHERE type = 'table'`. Print each name, indented by two spaces, under a heading. Empty line.

(b) `SELECT COUNT(*) FROM heroes` and the same for quests. Each gives one row with one value: `.fetchone()[0]`. Print both. Empty line.

(c) `SELECT * FROM heroes LIMIT 3`: print each row as it comes (a tuple).

EXPECTED OUTPUT

```
Tables in the Vault:
  heroes
  quests
  party
  regions

heroes: 120 rows
quests: 400 rows

('H001', 'Elowen Quickwater', 'Warrior', 'Human', 8, 'Harbourtown', '2018-10-20', 'yes')
('H002', 'Nerys Marchbank', 'Ranger', 'Dwarf', 3, 'Thornmere Keep', '2016-01-26', 'yes')
('H003', 'Linnet Oakhollow', 'Rogue', 'Elf', 5, 'Thornmere Keep', '2021-03-19', 'yes')
```

Hint: `*` means "every column". A row comes back as a tuple, in the order the columns were defined: id, name, class, kin, level, town, joined, active.

Exercise 2 The AutoFilter

10 min · `ex2_autofilter.py`

> **Raven from Captain Siv:** *"I need Rangers, level 12 or better, strongest first. Then every Troll we've beaten up north. Then who's retired, because they keep turning up to the feast."*

(a) `SELECT name, level, home_town FROM heroes WHERE class = 'Ranger' AND level >= 12 ORDER BY level DESC, name`. Loop with `for name, level, town in rows:` and print as shown. Empty line.

(b) Troll quests with outcome Success in the Northern Wilds **or** the Ironhills: quest_id, region, reward, biggest reward first. Put brackets round the OR. Empty line.

(c) Heroes whose `active` is not `'yes'` (`<>`), ordered by name. Print how many, then the first three names joined with `", "` and `...`.

EXPECTED OUTPUT

```
RANGERS OF LEVEL 12 AND UP
Wren Hawkridge       19  Thornmere Keep
Ysolde Goldenhand    18  Thornmere Keep
Maddox Northwind     14  Thornmere Keep
Wren Ashdown         14  Harbourtown
Idony Ravenscar      12  Thornmere Keep
Jory Northwind       12  Thornmere Keep
Wren Thistlewood     12  Mossgrove

TROLLS BEATEN IN THE NORTH
Q0231  Northern Wilds  1,800 cr
Q0086  Northern Wilds  1,710 cr
Q0370  Ironhills       1,610 cr
Q0212  Ironhills       1,570 cr
Q0247  Ironhills       1,250 cr
Q0315  Ironhills       1,230 cr

Retired heroes: 11
Doran Mossgrove, Doran Redfern, Fenwick Northwind ...
```

Hint: Text values go in **single quotes**: `'Ranger'`, and they must match exactly (`'ranger'` finds nothing). `AND` binds tighter than `OR`; without brackets, (b) would mean something else.

Exercise 3 Sharper filters

10 min · `ex3_sharper_filters.py`

> **Raven from Quartermistress Pell:** *"The Ironhills smiths will forge for Dwarves and Gnomes of level 12 and up, and only them. Who qualifies? Also the chapel keeps sending invoices for 'monster-related damage', and I'd like a count."*

(a) Heroes whose kin is `IN ('Dwarf', 'Gnome')` and whose level is `BETWEEN 12 AND 20`: name, kin, class, level, highest level first, then by name. Empty line.

(b) Two counts with `LIKE`: quests with "chapel" anywhere in the title (`'%chapel%'`), and quests whose title starts with "Clear " (`'Clear %'`). Empty line.

(c) How many quests have no region (`region IS NULL`)? And how many ended neither in Success nor in Fled (`NOT outcome IN (...)`)?

EXPECTED OUTPUT

```
Ysolde Goldenhand   Gnome  Ranger    18
Ada Fallowfield     Dwarf  Bard      17
Silas Goldenhand    Dwarf  Warrior   14
Bram Saltmarsh      Dwarf  Warrior   12
Maren Larkspur      Dwarf  Mage      12
```

```
Ninian Hawkridge    Dwarf  Warrior   12

Quests at a chapel: 34
Quests that start with 'Clear': 71

Quests with no region: 0
Quests neither won nor fled: 59
```

Hint: `%` in a LIKE pattern means "any text, or none". The Vault is tidy: no missing regions. Remember `IS NULL` anyway; next lesson the Vault won't be tidy, and `= NULL` never matches anything.

Exercise 4 Top of the board

10 min · `ex4_top_of_the_board.py`

> **Decree from Queen Aldara:** *"The five richest notices ever posted. The five quickest victories. And a list of everything we fight, in alphabetical order, because a Privy Councillor asked if we 'do dragons' and nobody could answer with confidence."*

(a) quest_id, monster, reward: `ORDER BY reward DESC LIMIT 5`. Empty line.

(b) quest_id, days, reward of successful quests, fewest days first and, for equal days, the bigger reward first. Five of them. Empty line.

(c) `SELECT DISTINCT monster FROM quests ORDER BY monster`: print how many, then all of them on one line.

EXPECTED OUTPUT

```
THE FIVE RICHEST NOTICES
Q0015  Wyrm    4,540 cr
Q0346  Wyrm    3,530 cr
Q0163  Wyrm    3,420 cr
Q0049  Wyrm    2,960 cr
Q0341  Wyrm    2,530 cr

THE FIVE QUICKEST WINS
Q0272  3 days     310 cr
Q0178  3 days     130 cr
Q0184  4 days     360 cr
Q0298  4 days     330 cr
Q0353  4 days     320 cr

8 monsters: Bandits, Ghouls, Giant Rats, Giant Spiders, Goblins, Trolls, Wolves, Wyrm
```

Hint: `ORDER BY days, reward DESC` sorts by days (ascending, the default), then by reward within equal days. Like Excel's Sort dialog with "Add Level". The answer to the Councillor is: yes, one kind, and it's going badly.

Exercise 5 Counting the hoard

10 min · `ex5_counting_the_hoard.py`

> **Raven from Quartermistress Pell:** *"The Royal Auditors arrive on Thursday. They want five numbers about the whole ledger and they want them before the biscuits run out."*

(a) One query, one row: `COUNT(*)`, `SUM(paid)`, `ROUND(AVG(reward), 2)`, `MIN(days)`, `MAX(days)`, `COUNT(DISTINCT region)`. Unpack the row into six names and print as shown. Empty line.

(b) The same kind of summary for the Wyrm only (add a `WHERE`): count, average notice, paid out.

(c) The average level of an active hero, rounded to 1 dp.

EXPECTED OUTPUT

```
Quests:          400
Crowns paid out: 146,150 cr
```

```
Average notice:  587.73 cr
Days in field:   3 to 14
Regions:         5

Wyrm: 7 quests, average notice 3,130.00 cr, paid out 1,970 cr
Average level of an active hero: 8.0
```

Hint: An aggregate squashes many rows into one value: COUNT, SUM, AVG, MIN and MAX are COUNTA, SUM, AVERAGE, MIN and MAX. Seven Wyrm notices worth 21,910 cr; 1,970 cr ever paid. Somebody tell Biscuit his cousins are expensive.

Exercise 6 The PivotTable, in SQL

10 min · `ex6_group_by.py`

> **Raven from Captain Siv:** *"Same table as last week, but out of the Vault this time: every monster, how many quests, the average notice, the total paid. Then the classes that average above level 8. Then which towns can field 20 active heroes."*

(a) `GROUP BY monster` with `COUNT(*)`, `ROUND(AVG(reward))`, `SUM(paid)`, ordered by `SUM(paid) DESC`. Print a header line and one row per monster. Empty line.

(b) Classes with `HAVING AVG(level) > 8`: class, count, average level (1 dp), highest average first. Empty line.

(c) Active heroes only (`WHERE`), grouped by home town, keeping towns `HAVING COUNT(*) >= 20`, most first.

EXPECTED OUTPUT

```
MONSTER         QUESTS  AVG NOTICE   PAID OUT
Bandits             72         553     30,080
Ghouls              61         722     27,710
Giant Spiders       42         834     26,060
Trolls              31       1,309     20,870
Goblins             81         313     18,480
Wolves              64         349     16,060
Giant Rats          42         145      4,920
Wyrm                 7       3,130      1,970

Classes with an average level above 8:
  Bard     13 heroes, level 9.7
  Ranger   21 heroes, level 8.7
  Warrior  32 heroes, level 8.6
  Cleric   16 heroes, level 8.2

Towns with at least 20 active heroes:
  Thornmere Keep  44
  Harbourtown     25
  Mossgrove       22
```

Hint: `WHERE` filters rows **before** grouping; `HAVING` filters groups **after**. The order is always SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT. Compare (a) with lesson 10, Exercise 2: same idea, different language.

Exercise 7 Worth it? STRETCH

15 min · ex7_worth_it.py

> **Raven from Captain Siv:** *"Yes or no: which monster should my best parties hunt? Crowns earned per day in the field, all quests together. Only monsters we've fought at least 30 times; I don't trust small numbers. Then the Wyrm on its own, so I can show the Queen."*

(a) Per monster: `COUNT(*) AS quests` and `ROUND(SUM(paid) * 1.0 / SUM(days), 1) AS per_day`, `HAVING quests >= 30`, `ORDER BY per_day DESC`. Empty line.

(b) For the Wyrm: number of quests, number won (a second query), crowns per day. Empty line.

(c) Let the Vault pick the winner: the same grouping, `ORDER BY` the per-day sum, `LIMIT 1`, and print Siv's answer.

EXPECTED OUTPUT

```
CROWNS PER DAY IN THE FIELD (monsters with 30+ quests)
Trolls           31 quests   75.6 cr/day
Giant Spiders    42 quests   71.8 cr/day
Bandits          72 quests   53.9 cr/day
Ghouls           61 quests   52.5 cr/day
Wolves           64 quests   34.1 cr/day
Goblins          81 quests   32.7 cr/day
Giant Rats       42 quests   17.6 cr/day

Wyrm: 7 quests, 0 won, 28.6 cr/day

Captain Siv's answer: send the best parties after the Trolls.
```

Hint: `* 1.0` matters: SUM by SUM is whole number by whole number, and SQLite then drops the decimals. In lesson 10 the Trolls came out at 79.0 per day: that was the average of each quest's own rate. This is total crowns over total days. Both honest, slightly different questions.

Bug hunt Pages from the Grimoire, page 11

15 min · bughunt/vault_report.py

Folded inside the lid of the oldest chest in the Vault: the hard quests (difficulty 3 and up) by region, with wins and the average payout. **Three** bugs: one crashes (a syntax error near "WHERE": the clauses have a fixed order), one quietly finds no wins at all, one gets every average a few pennies wrong. One commit per bug, each message explaining what was wrong and why.

EXPECTED OUTPUT ONCE FIXED

```
HARD QUESTS (DIFFICULTY 3+), BY REGION
REGION           QUESTS   WON    AVG PAID
Ironhills            54    33   507.41 cr
Thornwood            54    32   431.30 cr
Northern Wilds       51    28   490.78 cr
Greyfen              49    23   301.63 cr
Saltmarsh            48    31   406.88 cr
```

More riddles. Five more torn pages wait in `bughunt/riddles/`, one bug each (the last one has two), getting harder as you go. Optional: for fast finishers, or a quiet evening.